Making Time-Sensitive Networking Deployable: A Comprehensive Lifecycle Architecture

Special Session Paper

Rubi Debnath[1], Paul Pop[2], Silviu S. Craciunas[2,3], Marc Boyer[4], and Sebastian Steinhorst[1]

[1]TUM School of Computation, Information and Technology, Technical University of Munich, Germany
[2]DTU Compute, Technical University of Denmark, Denmark
[3]NXP Semiconductors, Vienna, Austria
[4]DTIS, ONERA, Université de Toulouse, 31000, Toulouse, France
[1]firstname.lastname@tum.de, [2]paupo@dtu.dk, [3]silviu.craciunas@nxp.com, [4]marc.boyer@onera.fr

***Abstract*—Time-Sensitive Networking (TSN) provides deterministic guarantees and bounded latency for safety-critical applications across different domains. While the IEEE 802.1 TSN standards provide a comprehensive set of mechanisms for time synchronization, shaping, scheduling, resource reservation, and reliability, deploying TSN in real-world applications remains a complex, multi-step engineering task. Existing research primarily focuses on optimizing individual objectives, such as increasing schedulability, reducing latency, accelerating solution generation, or improving performance analysis. However, whether these extensively researched solutions can be deployed in real-world systems remains unanswered, as the assumptions made in these methodologies often differ from practical hardware constraints. Therefore, a broader, holistic view of end-to-end TSN deployment and management remains an open research question. Currently, there is a lack of a unified perspective, consensus, and coordinated research efforts focusing on a comprehensive end-to-end TSN deployment workflow. We present a comprehensive overview of the TSN deployment lifecycle, current challenges, limitations of existing tools, and future research directions for TSN deployment and management. Lastly, we identify key research gaps from a deployment perspective and provide guidance for the development of next-generation deployable TSN networks.**

***Index Terms*—Time-sensitive networking, TSN, configuration, deployment, automation, LLM, AI.**

I. Introduction

Time-Sensitive Networking (TSN) is a set of IEEE 802.1 sub-standards that provides deterministic communication capabilities, including time synchronization, bounded latency and jitter, resource reservation, and reliability mechanisms. TSN has a wide range of applications, including automotive systems [1], industrial automation [2], spacecraft [3], and healthcare [4]. To support different application requirements, TSN provides multiple scheduling and shaping mechanisms, including Time-Aware Shaper (TAS) standardized by IEEE 802.1Qbv [5], Cyclic Queuing and Forwarding (CQF) [6], Cycle Specific Queuing and Forwarding (CSQF) [7], Multi-CQF [8]–[10], Credit-Based Shaper (CBS) [11], Asynchronous Traffic Shaper (ATS) [12], [13], and Frame Preemption (FP) [14], [15]. Among these, TAS has been extensively studied because it enables deterministic transmission of Time-Triggered (TT) flows, also referred to as Scheduled Traffic (ST), using a Gate Control List (GCL). However, the generation of GCLs is an NP-hard problem [16], and the complexity increases with the number of flows and network size [17].

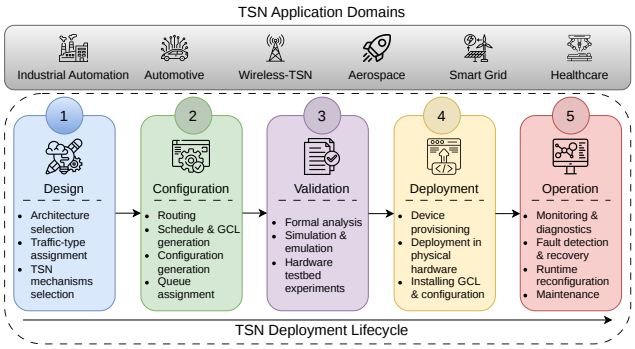


Fig. 1: Overview of the TSN deployment lifecycle discussed in this paper. The lifecycle consists of five sequential stages: design, configuration, validation, deployment, and operation.

Although significant progress has been made in TSN schedulability analysis, configuration, and optimization, deploying TSN in real-world systems remains a complex multi-step engineering challenge. Existing research primarily focuses on individual optimization problems, e.g., improving schedulability [10], reducing latency [9], accelerating configuration synthesis, or performing worst-case delay analysis [18], [19]. These approaches have significantly advanced the theoretical foundations of TSN. However, whether solutions developed in research can be directly integrated into real-world deployments remains an open question. The assumptions made by optimization and analysis methods often differ from practical deployment constraints, including hardware limitations, configuration interfaces, device capabilities, and operational requirements.

Practical TSN deployment is not limited to solving a scheduling problem or finding a suitable configuration. A system engineer must determine the appropriate TSN architecture, select suitable combinations of mechanisms, assign traffic types, synthesize schedules, configure network devices, validate the resulting configuration, deploy it to physical infrastructure, and continuously manage the network during operation. In addition, TSN networks support mixed-shaper architectures, where multiple mechanisms such as TAS, CBS, ATS, and CQF coexist at the same egress port. While such combinations can provide improved flexibility, they also in-

troduce additional interactions between mechanisms, increasing the complexity of configuration and verification [15], [20]–[23]. Moreover, selecting the appropriate combination of mechanisms from a pool of different shapers and schedulers for a given input becomes a crucial design choice.

Therefore, TSN deployment should be considered as an end-to-end lifecycle rather than as a collection of isolated optimization problems (refer to Fig. 1). Each stage represents a set of decisions required to transform application requirements into an operational TSN network. The output of each stage becomes the input for the subsequent stage. Therefore, the decisions made during early stages directly influence later validation, deployment, and runtime management.

To bridge the gap between TSN research and practical deployment, this paper presents a comprehensive lifecycle-oriented analysis of TSN deployment and management. The main objective of this paper is to establish a unified perspective on deployable TSN networks and highlight the research directions required to move from TSN mechanism optimization toward complete end-to-end TSN deployment workflows.

This paper starts by discussing the background of the TSN deployment lifecycle (refer to Section II), highlights the challenges of TSN deployment and runtime operation (refer to Section III), discusses the gaps and limitations of current TSN profiles and standardization efforts (refer to Section IV), and presents runtime reconfiguration challenges and deployment-oriented solutions (refer to Section V). Finally, we discuss automation-aided TSN deployment and the role of Artificial Intelligence (AI) and Machine Learning (ML) in TSN management and configuration (refer to Section VI).

II. TSN Deployment Lifecycle: From Design to Operation

The TSN deployment lifecycle consists of five sequential stages, in which the output of one stage is used as the input to the subsequent stage. Fig. 1 presents the different stages involved in end-to-end TSN management and deployment. We broadly categorize these stages into five phases: (1) design, (2) configuration, (3) validation, (4) deployment, and (5) operation. Each of these stages is highly complex and involves ongoing research on optimization. Therefore, the overall configuration space is large. Moreover, the configuration process is still poorly understood by the community. System engineers must navigate numerous interdependent decisions while accounting for the hardware limitations of commercial switches and satisfying end-to-end timing requirements. Multi-domain and wireless TSN networks, as well as runtime reconfiguration, further increase this complexity. Current tools are sparsely developed, are rarely open sourced, and seldom scale. Finding a feasible configuration for medium-scale networks requires person-months of engineering effort. In addition, precise time synchronization remains a fundamental requirement for many TSN mechanisms. Therefore, large-scale industrial deployment requires addressing several challenges across the TSN deployment lifecycle [2]. We discuss these stages below.

A. Design Phase

In the design stage, the requirements and architecture of the TSN network are defined before configuring the network devices. Here, network designers must determine the appropriate combination of TSN mechanisms and perform traffic-type assignment (TTA) based on the quality-of-service (QoS) requirements. There is no universal solution applicable to all domains. Therefore, different applications, such as industrial automation, automotive, and aerospace systems, require customized solutions based on their specific requirements. Previous work has used metaheuristics [24] and Deep Reinforcement Learning (DRL)-based [25] solutions to solve the isolated TTA problem.

B. Configuration Phase

The configuration stage is one of the most challenging, as the standard leaves the configuration process to the implementation. This stage requires configuring several parameters for a simple TSN network, including time synchronization, schedule generation, queue assignment, cycle duration selection, bandwidth (BW) reservation, redundancy, and/or resource allocation. For TAS, optimized algorithms have been developed to generate the GCL [16], [26], [27]. For CQF, CSQF, and Multi-CQF, cycle duration selection, BW distribution, and configuration generation become crucial [9], [10], [28]–[31].

C. Validation Phase

Before deploying the network in real-world scenarios, thorough validation of the design and configuration is necessary to avoid network downtime. The validation stage ensures that the generated TSN configuration satisfies the QoS requirements before deployment in an operational environment. Existing research employs multiple validation approaches, including analytical methods, simulation frameworks [32], formal verification techniques, and physical testbeds [33]. Analytical methods provide Worst-Case Delay (WCD) estimation and schedulability analysis of the flows in the network using tools such as Network Calculus (NC) [19], [20]. Formal verification techniques, such as SMT and ILP, provide WCD estimates. Nevertheless, NC-based approaches are often highly pessimistic, and formal mathematical methods may not fully capture real-world scenarios. Therefore, simulation-based validation [15], [22], [32], [34] is used to complement NC and formal verification approaches. Since simulation results may not fully represent hardware implementation scenarios [33], hardware testbeds remain important for validation.

D. Deployment Phase

In the deployment phase, the validated configuration is transferred to the physical network infrastructure. A desirable future TSN capability is an automated one-touch or one-click deployment solution for the entire network, avoiding device-by-device configuration and simplifying large-scale cross-domain deployments. Automated support for debugging and configuring network devices is also highly desirable.

E. Operation Phase

The operation stage consists of the continuous management and monitoring of an operational TSN network. The goal is to monitor network logs, detect malicious activities, adapt the network according to new flows or faulty nodes, and/or detect timing violations. Another major need is the dynamic reconfiguration during runtime for networks under operation.

III. Why TSN Remains Difficult to Deploy

TSN deployment is more than a technical problem. In multi-domain systems, domains may be operated by different teams, departments, or companies, creating a systems-of-systems challenge that includes governance, ownership, and rules defining who may change each configuration and when. Although these concerns affect deployability, this paper focuses on the technical challenges. Industrial applications carry flows with widely varying timing requirements. A control loop may require short and nearly constant latency, whereas alarm, video, and diagnostic traffic are bursty and primarily require BW. Placing every flow under the TAS mechanism is unnecessary, and for large networks, it is unrealistic. A more suitable architecture combines TAS with shapers such as CBS, ATS, or CQF, where TAS carries tightly constrained periodic streams and the other shapers serve traffic for which a fully TT schedule may be too expensive to configure [21]. The difficulty with such combined mechanisms is that the shapers interact. Closing a TAS gate changes the service received by the CBS and ATS queues on the same egress port, and every queue or BW fraction assigned to one shaper becomes unavailable to the others. We are therefore interested in a joint configuration problem that decides and optimizes TTA, routing, queue assignment, GCL, and shaper parameters while considering the interdependencies [23].

Next, consider a flow that crosses several domains. Each domain may find a feasible local schedule, while the complete path can still miss its end-to-end deadline. Time is spent at domain boundaries, synchronization errors must be accounted for, and the sum of the local latency budgets must remain within the end-to-end requirement. The domains may also differ in cycle lengths, shapers, and approaches to timing analysis. In addition, an administrative domain may be willing to disclose a delay bound and the capacity it offers, but not its topology or complete stream set. A centralized controller has more information to work with at the cost of a larger configuration problem and reduced domain autonomy. Hierarchical approaches instead exchange smaller abstractions; however, an abstraction may hide a feasible configuration [35]. Based on these observations, we formulate our first finding.

Finding 1: What is missing is a timing interface that is sufficiently detailed for end-to-end analysis, while not forcing a domain to reveal its internal configuration.

A further limitation of TSN configuration (refer to Fig. 2) is the assumption that configuration occurs once, at design time. A deployed network does not remain fixed. Network devices are added, application modes are changed, links fail, and flows are added or removed. In such cases, waiting hours for a new global configuration is not an option. The admission or recovery algorithm must find a solution within a reasonable time and modify the existing configuration as little as possible. This is particularly challenging in multi-shaper networks, since moving a single TAS window may change the WCD of the ATS or CBS flows that share the egress port, and an update to an inter-domain flow may force several controllers to revise their local configurations. Incremental methods, however, reuse the previous solution and can admit changed flows much faster than a complete recomputation [36], although the transition between the old and new configurations still needs to be analyzed [23]. This is discussed in further detail for the TAS mechanism in Section V.

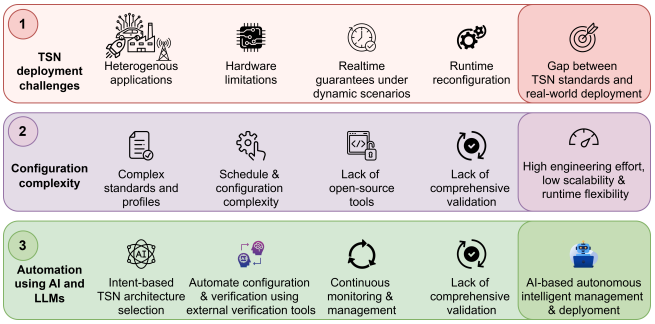


Fig. 2: Roadmap and challenges towards end-to-end TSN deployment and management.

On the other hand, TSN hardware imposes its own limitations. A solver can produce a mathematically valid schedule that the selected switches cannot store or execute, because egress ports have a limited number of queues and finite buffer memory. The GCL holds a bounded number of entries, and gate events can only be placed at the timing resolution supported by the device. When several shapers share a port, these limits become even more restrictive, since TAS, CBS, ATS, and CQF compete for the same queues and buffers, and shaper implementations differ in hardware cost [37]. Devices from different vendors may additionally support different parameter ranges or subsets of the standards. These hardware capabilities should be included as constraints in the problem formulation. Doing so may increase the size of the formulation, but omitting these constraints can result in undeployable configurations. These considerations motivate our second finding.

Finding 2: Hardware capabilities should be provided as inputs to configuration synthesis rather than being treated as checks applied after the synthesis step.

Another challenge lies in the existing evaluation methodologies. The evaluation practice itself needs to evolve. Current evaluations report end-to-end delays and algorithm runtimes and stop there. These metrics provide limited insight into deployability in real-world systems. A schedule may satisfy all deadlines in the model and still not fit within the eight queues of an egress port or the GCL length of the respective switch, making the entire schedule unusable. The TSN community should additionally report the amount of device resources occupied by a configuration (queues, buffers, and GCL entries), the time required to admit or update a stream, and the number of devices and existing streams affected by such an update. Multi-domain studies should further specify their synchronization assumptions and the information exchanged between controllers. A configuration that is feasible in an abstract model is not necessarily one that can be installed, updated, and analyzed on the target network. The reported metrics should allow system engineers to understand this difference. Closing this abstraction gap requires shared benchmarks, constraints,

best practices, and guidelines established through standards or organization-specific case studies. These artifacts can reduce assumptions and support consistent implementation. These aspects become particularly important as TSN profiles provide deployment guidance for specific application domains. However, several fundamental questions regarding mechanism selection, parameter configuration, and configuration validation remain open, motivating a closer examination of the current industrial TSN profiles and their limitations.

IV. Standardization Gaps Toward Deployable TSN

TSN profiles specify which features, options, defaults, and procedures apply within an application domain [38]. Some profiles, like the automotive one, also include recommendations for configuring optional parameters [39]. As outlined in Section III, TSN profiles are intended to reduce the configuration space by providing concrete guidelines to the system engineer. Using the two recently published profiles, IEC/IEEE 60802 [38] and IEEE 802.1DG [39], as our reference, we highlight four practical questions: (i) *which mechanism or combination of mechanisms satisfies the application requirements*, (ii) *which parameter values should be assigned*, (iii) *how should the resulting configuration be verified*, and (iv) *whether the profile covers all use cases within its domain*.

The profiles provide the selection of features and options while deferring most concrete configurations and recommendations to system integrators, with synchronization in 60802 being a notable exception [38], [39]. The profiles answer what a device vendor should implement rather than the system integrator's question about how to use it (see Table I).

A. Mechanism Selection Gap

IEC/IEEE 60802 defines eight traffic types, two conformance classes, and normative synchronization budgets of 1 μs and 100 μs over up to 100 hops for the working clock and the global time, respectively [38]. The quantified traffic characteristics from the underlying use-case documents [40], [41] are retained only as typical ranges or qualitative categories. Neither IEC/IEEE 60802 nor IEEE 802.1DG provides a comprehensive mandatory (normative) mapping from traffic types to concrete TSN mechanisms. IEEE 802.1DG binds mechanisms to traffic classes, but the class to which a given application stream belongs remains the integrator's decision. IEC/IEEE 60802 explicitly preserves this freedom, stating that ST is only one means of achieving time-triggered transmission and that its traffic-type categories permit different feature selections to reach the same goal [38]. An Industry IoT Consortium (IIC) white paper maps traffic types to device capabilities but identifies itself as input to the standards, places configuration generally out of scope, and leaves even the choice between strict priority (SP) and ST for cyclic streams unspecified [42].

IEC/IEEE 60802 adopts a revised variant of the taxonomy but not the mapping, offering a traffic-class mapping only as an example [38]. In contrast, IEEE 802.1DG mandates ingress policing, together with CBS on the two highest traffic classes and ATS on at least one traffic class [39]. TAS and FP are optional, while Frame Replication and Elimination for Reliability (FRER) [43] is not used. Furthermore, time synchronization is only recommended and becomes mandatory only when the optional ST is supported, with five example synchronization mechanisms listed for the system integrator [39].

Table I: Engineering decisions vs. what the two TSN profiles resolve (● resolved normatively, ◑ partially addressed or informative guidance only, ○ open or explicitly delegated).

Engineering decision	60802	802.1DG
Traffic types and timing requirements	◑	◑
Synchronization accuracy and error budgets	●	○
Mandatory device feature set (conformance)	●	●
Mechanism selection per traffic type	◑	◑
Shaper and schedule parameter values	○	◑
Configuration model and synthesis toolchain	◑	○
Runtime reconfiguration	◑	○
End-to-end configuration verification	○	○

Finding 3: Current TSN profiles do not provide a definitive mapping between application requirements, traffic types, and appropriate TSN mechanisms. Consequently, mechanism selection remains largely dependent on the expertise of system integrators.

We argue that the selection gap persists because TSN mechanisms overlap in functionality and experts optimize for different objectives, such as tight latency and jitter guarantees, configuration simplicity, low adoption risk, or hardware cost, with no single mechanism being optimal for all objectives. Disagreement is therefore not only a matter of incomplete understanding, as standardizing on a mechanism that best serves one objective can make other objectives harder to achieve. For the highest-priority class in the absence of overload, CBS and ATS shaping do not improve the worst-case latency but instead smooth the shaped class's bursts and improve the average latency of lower-priority classes [44]. Neither profile distinguishes between worst-case guarantees and burst regulation, leaving system integrators to infer what each mandated mechanism actually contributes. Moreover, configuring CBS according to the standard's own idle-slope rule can cause worst-case analysis to fail for most configurations that are in fact schedulable [45]. ATS, meanwhile, provides only limited, load-dependent advantages over CBS [20]. Together, these observations further complicate the selection of the appropriate shaping mechanism or combination of mechanisms.

Experimental results also reveal significant differences among mechanisms. In one study on automotive 100BASE-T1 hardware, a low-priority interferer increases the measured two-hop worst-case latency from approximately 40–50 μs to 289 μs under SP and to 269 μs under CBS. Under TAS, however, the latency stays at 47 μs [46]. Earlier work likewise identified the shaping mechanism for control traffic as an open question, warning that an unstructured collection of mechanisms could drive adopters back to FlexRay or CAN FD [47].

IEC/IEEE 60802 requires TAS at 100 Mbit/s and 1 Gbit/s in its feature-rich conformance class. It also requires FP, unconditionally for bridges and conditionally for end stations, but does not reference ATS [38]. Domain differences alone do not fully explain the divergence between the mechanism choices of the two profiles. Within a single year, contributions

to the automotive group recommended CBS with TAS and preemption as last resorts [48], preemption before TAS due to risk considerations [49], and a hardened AVB subset containing neither [50]. Some training material even argues that TAS and CQF have no role in a converged in-vehicle backbone network [44]. Formal bounds exist for all mechanisms, but TAS additionally enables joint task/network scheduling to minimize alignment delays and tightly control jitter [51].

Whether a profile covers all target-domain use cases remains a practical question. Between 2019 and 2022, at least three incompatible traffic-type taxonomies were proposed [52]–[54], before the published IEEE 802.1DG profile settled on an informative characterization using order-of-magnitude figures [39].

The asynchronous foundation of IEEE 802.1DG assumes independently developed domains that exchange rate contracts without a common clock [55]. Software-defined vehicles centralize integration, and automated driving in particular may require a common notion of time for sensor fusion [1], [56]. Moreover, the cause-effect chains of ADAS/AD functions accumulate sampling delays at every task and network boundary [57], [58], and their host systems are highly utilized [59]. Asynchronous mechanisms can bound such chains, but minimizing them requires co-scheduling the task and network domains [51], a capability that only TAS provides. The profiles have thus not resolved the selection problem but have only institutionalized it.

B. Parameterization Gap

Even after the mechanism set is fixed, deriving suitable parameters remains a difficult task for the system integrator. IEEE 802.1DG provides closed-form policing parameters, with an idle slope equal to the sum of the stream rates [39]. However, this corresponds to the same allocation under which the worst-case analysis fails [45]. Beyond this suggested policing rule, neither profile specifies how scheduler and shaper parameters should be derived from application requirements. Such derivation methods are valid design choices, and we do not argue that a profile should mandate a specific method. Instead, what is missing is a higher-level framework that provides a standardized way to express the requirements that a configuration must satisfy and a means to check the result independently of the tool that produced it. IEC/IEEE 60802 demonstrates that this separation is feasible by standardizing device capabilities in the form of a digital data sheet while leaving open the computation that uses these capabilities [38].

Finding 4: The selection of a TSN mechanism alone does not determine network guarantees. The resulting performance depends on parameter configuration, traffic characteristics, and interactions between mechanisms.

For TAS, GCL synthesis is NP-hard [16], and exact formulations reach their practical limits on moderately sized networks [17]. Nevertheless, tailored heuristics can schedule instances with 2000 nodes and more than 10 000 flows within one hour [60], and systematic experimental evaluations have studied the schedulability and scalability of such approaches [61]. An in-vehicle backbone consisting of a few switches is therefore orders of magnitude smaller than these evaluated instances. Design-time guarantees for CBS or ATS also require assigning streams to at most 8 traffic classes, which is itself a discrete optimization problem [24], and deriving the coupled shaper parameters. Together with subsequent worst-case analysis, for which computing tight bounds can be NP-hard [62], the overall problem remains computationally complex. Selecting an asynchronous mechanism therefore shifts complexity from GCL synthesis to verification. Where no analysis is mandated, this complexity is effectively dropped rather than solved. This tradeoff motivates Finding 5.

Finding 5: Choosing simpler mechanisms does not remove complexity. It only shifts complexity from schedule synthesis to worst-case analysis and verification.

The complexity warnings in IEEE 802.1DG concern only features that are optional or not used, namely ST and the stream identification used for frame replication [39], while IEC/IEEE 60802 mandates TAS without discussing its complexity [38]. Moreover, the complexity argument against TAS in [44] is based on work that also proposes heuristics to make the synthesis tractable (see [63]). Thus, the complexity that influences mechanism selection is documented by research but is not weighed symmetrically by the two profiles. Both profiles define certain conformance floors, such as minimum queue and stream-filter counts, but neither guarantees a minimum capacity for GCLs. IEC/IEEE 60802 limits gate granularity but treats list lengths and cycle times as self-declared datasheet capabilities [38]. Similarly, the entry counts in IEEE 802.1DG are provided only as guidance and, as the standard states, do not constitute conformance requirements [39]. These challenges motivate Finding 6.

Finding 6: TSN profiles leave parameter values, optimization, and deployment decisions largely implementation-dependent.

The question of configuration responsibility is also only partially addressed. IEC/IEEE 60802 adopts the fully centralized model of 802.1Qcc [64], requiring NETCONF/YANG management. Resource-constrained devices without native NETCONF support can only be integrated through proxies using user-specific protocols [38]. However, the computation of schedules and paths performed by the Centralized Network Configuration (CNC) entity remains implementation-dependent. Its interface to engineering tools is user-specific [38], while IEEE 802.1DG specifies no configuration protocol at all [39]. Consequently, the actual computation of the network configuration remains outside the scope of the standards. Although IEC/IEEE 60802 mandates candidate-based configuration updates with rollback and isolates newly added stations until the CNC admits their traffic, the method used to compute incremental updates remains unspecified.

C. Configuration Verification Gap

Conformance testing certifies devices, not network configurations. The common test specification currently being developed for IEC/IEEE 60802 defines its testing scope at the level of stations and their components [65], and the certification programs built on that baseline inherit the same scope. They do not provide a standardized method to verify

whether a particular assignment of streams, parameters, and schedules satisfies application-level requirements (Finding 7).

Finding 7: Device conformance has a clear owner, i.e., the vendor that tests a device against a specification. A deployed configuration spans multiple mechanisms, vendors, and integrators, leaving no single party responsible for its verification and no current program covering it.

The analytical guidance provided by current standards is also incomplete. Even the bound provided by the AVB profile does not cover its worst-case traffic pattern [44], [66]. The CBS analysis in 802.1Q Annex L excludes any interruption to BW availability caused by gate operation [67], and IEEE 802.1DG declines to give guidance on how such combinations affect network behaviour [39].

Finding 8: No standardized instrument validates whether a complete configuration satisfies the intended guarantees.

The selection and parameterization gaps share a common root, which may also explain the expert disagreement. Guarantees depend on parameter values rather than on the mechanism alone. Therefore, the configuration space grows exponentially with the number of parameters rather than with the number of permitted mechanisms. These observations reveal three needs, none of which requires a profile to mandate a mechanism:

1) **Methodology is required:** Validated configuration patterns that map traffic classes to mechanism combinations, together with the required analyses and hardware assumptions. The closest existing attempts [42], [68] are not normative documents. However, the ongoing IEEE project on shaper parameter settings [69] demonstrates that this need has been recognized.
2) **Tooling is required:** Open configuration synthesis approaches that consume the hardware model already standardized by IEC/IEEE 60802 and validate configurations rather than only devices. These requirements connect to Section III, Section V, and Section VI.
3) **Transparent guidance is required:** Published negative results identifying which mechanisms are substitutes, which combinations do not compose, and which mechanisms are unsuitable for specific use cases.

V. Toward a Deployment-Oriented Solution

Configuration does not end with the initial deployment. Even after selecting the optimal mechanisms and generating the initial configuration, there are several reasons why a TSN network must be reconfigured during operation. Streams may be added or removed, an application may switch its operating mode, a device may be replaced during maintenance, and, in most projects, the design itself is extended incrementally. In addition to these planned changes, the network must recover from link, port, or device faults at runtime. Such scenarios impose stringent requirements: computing and installing a new configuration must be completed within a bounded time, while traffic that does not rely on the failed component must continue to satisfy its timing and reliability requirements. One approach to reducing both computation and deployment effort is incremental reconfiguration, i.e., modifying as few streams and devices as possible. However, minimizing the number of affected elements alone is insufficient, as timing and loss guarantees must also be maintained throughout the transition from the old configuration to the new one [23], [70].

A. Closed Loop Runtime Reconfiguration

It is useful to view runtime reconfiguration as a loop consisting of four steps: *detect–compute–deploy–verify*. The detection step may appear straightforward, but it is not, as the system must determine whether a component has failed or a stream has changed its requirements, and identify which flows are affected before any further action can begin. Computation is the aspect most familiar to the scheduling community, i.e., selecting the admitted flows together with their routes, queues, shapers, GCLs, and parameters; however, the difference is that the algorithm now operates under the processing and memory constraints of the deployed system. The resulting configuration must then be deployed to the bridges and activated. In the final step, the controller verifies that each device has accepted its configuration, that the installed state is consistent across devices, and that the timing constraints continue to be satisfied. A fast scheduling algorithm provides limited benefit if fault detection is slow or if the update protocol can leave the network in an inconsistent state [70], [71]. These observations motivate Finding 9.

Finding 9: A fast scheduling algorithm alone is insufficient for runtime TSN reconfiguration. Practical solutions require an integrated *detect–compute–deploy–verify* workflow to ensure consistent updates and continued satisfaction of timing requirements.

B. Failure Recovery and Degraded Modes

Where should fault tolerance originate? The standards provide precomputed redundancy mechanisms. FRER (IEEE 802.1CB [43]) duplicates frames over disjoint paths, and backup configurations can be prepared for anticipated faults. Recovery is then fast; however, the BW, memory, and configuration resources required for the backups are consumed regardless of whether a fault occurs. The alternative is to reroute and reschedule online after the failure has been detected; resources are then consumed only when needed, at the cost of bounding the computation and deployment times. Neither approach is sufficient on its own. The number of backups grows combinatorially with network size and the number of faults to be tolerated, while after a severe fault, the remaining network may no longer provide the nominal level of service. Kong et al. [72] study runtime recovery of TT traffic under such conditions. When available resources are insufficient, the recovery algorithm either maintains full functionality, moves selected applications into degraded mode, or rejects lower-utility traffic. This utility should be modeled separately from the 802.1Q priority, which also influences schedulability [70]. These tradeoffs motivate Finding 10.

Finding 10: Precomputed redundancy and online recovery alone are insufficient for scalable TSN fault tolerance. Deployable TSN networks require resource-aware fault tolerance strategies that dynamically balance reliability, resource availability, and application criticality.

C. Consistent Updates

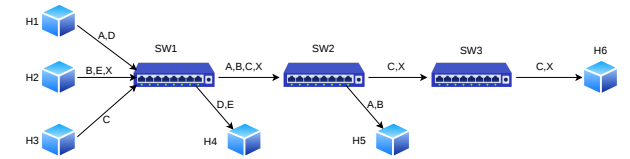
A feasible new configuration does not define the transition toward that configuration. Bridges activate routing tables and GCLs at different times, while frames from the previous configuration may still be queued. Such frames may encounter stale forwarding states and be dropped at an intermediate bridge, or may occupy a TAS window assigned to another stream, delaying subsequent frames and violating the schedule. An update method must hence define the order and activation times of device changes while accounting for in-flight and queued frames. The necessary guarantees must cover the transition interval as well as the two steady-state configurations. Conflict-free update methods address this problem for scheduled flows [71]. However, when routing, multiple shapers, and device-specific configuration interfaces change simultaneously, consistent deployment remains a challenge [70]. These requirements motivate Finding 11.

Finding 11: Consistent TSN deployment requires update mechanisms that coordinate configuration changes across devices and preserve timing guarantees throughout the transition between configurations.

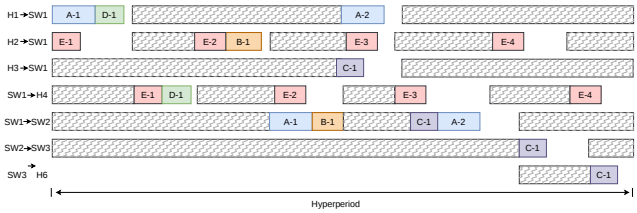
D. Design Point Solution

Reconfiguration capabilities can also be incorporated into the initial schedule. The Windows Precedence Exclusion (WPEx) method introduces this concept for TAS [73]. WPEx assigns sets of frames to precedence-separated windows and reserves slack by spreading and enlarging the windows. Admitting or rerouting a stream then becomes a matter of finding a path whose sequence of windows still has available capacity. The GCL remains unchanged, only the routing and stream-to-queue state are updated. Fewer device configuration elements are modified, reducing the exposure to inconsistent GCL activation. A WPEx schedule also tolerates missing frames, which is relevant when a stream is removed or interrupted by a fault. Under protective gating, lower-priority traffic may use the unused portion of an enlarged window left by TAS frames, subject to the assumed priority and arrival conditions [74]. Figure 3 illustrates the topology and the two steps: the enlarged windows with their reserved slack and the admission of a new stream X into that slack (shown in green).

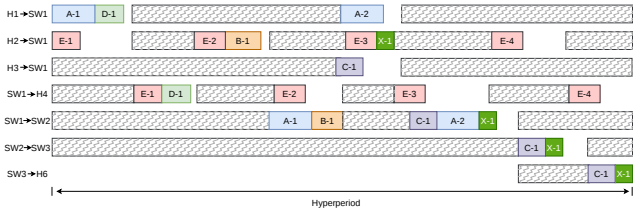
WPEx represents one possible design approach in this direction, addressing a specific problem under specific assumptions. A WPEx-compatible TAS schedule must exist as the starting point, and admission succeeds only if the selected path still has sufficient slack. This approach assumes a centralized configuration entity, leaving fault detection and fault tolerance of the controller out of scope, and relies on consistent updates of routing tables even though the GCL remains unchanged. In this approach, we have not evaluated whether it extends to ports where multiple shapers interact, to paths that cross administrative domains, or to devices with heterogeneous hardware capabilities. Together with flexibility-based admission [75], WPEx demonstrates that an initial schedule can reserve resources and structure that reduce the cost of future changes. Runtime TSN reconfiguration is therefore a control loop whose phases require bounded execution times and validation mechanisms, rather than merely a scheduling problem to be solved more quickly. Several challenges remain open, as summarized in Finding 12, some of which may be addressed through ML-based automation.



(a) Topology; the edge labels list the routed streams, including the new stream X.



(b) Windows enlarged to provide slack.



(c) Stream X admitted using the available windows.

Fig. 3: WPEx example: (a) the topology; (b) enlarged TAS windows provide slack; (c) a new stream X uses the available windows without changing the GCL.

Finding 12: Incremental timing analyses that account for interacting shapers are largely absent, failure-aware coordination among domain controllers remains unresolved, and update protocols that preserve guarantees during deployment are still in their early stages.

Automated management has a natural role in addressing these challenges, provided that the roles and responsibilities remain clearly separated. An automated agent may propose recovery actions, coordinate the reconfiguration workflow, and drive the configuration tools, while deterministic timing analysis and device-level validation determine whether a proposed action can be activated. The large language model (LLM)-assisted architecture presented in the next section is built around precisely this separation, combining AI-assisted tools with deterministic verification to preserve correctness and timing guarantees.

VI. AI-Aided TSN Deployment

The configuration and management challenges discussed in the previous sections motivate the exploration of learning-based approaches for TSN deployment. As network size and configuration complexity increase, exploring the configuration

space requires evaluating a large number of candidate solutions while accounting for routing, scheduling, traffic assignment, and hardware constraints. Formal verification techniques remain essential for providing deterministic guarantees, but repeatedly applying them during reconfiguration for search space exploration can become a major computational bottleneck [76]. Graph Neural Networks (GNNs) are particularly promising in this context because they can represent network topology [77]. Recent advances in deep reinforcement learning (DRL) have demonstrated their potential for TSN routing and scheduling [29], [30], [78], and TTA [25] in complex network scenarios. Moreover, generative AI and LLMs have emerged as a promising tool to support system engineers with configuration and debugging. [79] surveys the advantages and roles of LLMs in intelligent network management. Prior work has explored using LLMs to extract information from complex network documents [80], and has proposed a verified programming method for LLMs to generate router configurations [81]. TSN management and configuration are largely performed manually by domain experts and system engineers, requiring significant time and effort and carrying a high risk of errors from human judgment failures. LLMs can improve efficiency here by detecting errors and monitoring network logs in real time [79], and by learning TTA, traffic patterns, TSN architectures, and other design information to dynamically suggest changes to system engineers based on network conditions [79]. These opportunities motivate Finding 13.

Finding 13: LLMs can meaningfully support multiple stages of the end-to-end TSN deployment lifecycle (refer to Fig. 4), from design to validation and troubleshooting.

However, using LLMs for network configuration is not straightforward and presents additional challenges [82]. LLMs cannot magically replace system engineers by fully automating management and configuration. Rather, they can meaningfully reduce the overall time spent on TSN deployment. The emerging capabilities of LLMs present an opportunity to explore their applicability across different stages of TSN management and orchestration and to identify the use cases in which they can provide meaningful benefits. As a first step, [83] proposed the first end-to-end TSN management framework. Building on this, [84] presented TSNBench, benchmarking 16 state-of-the-art LLMs on multiple-choice and open-ended TSN questions. Verification and approval of LLM-provided solutions remain at the discretion of domain experts. Each configuration or design choice is accompanied by a model-generated confidence score, helping the system engineer decide whether to accept the model's output.

To fully realize the potential of LLMs for TSN management and deployment, our community must advance several open research directions. As demonstrated in [84], off-the-shelf LLMs fail to assist with schedulability analysis and open-ended questions. The models, however, exhibit a good understanding and capability in solving multiple-choice questions related to TSN. This gap highlights clear directions for future research aimed at optimizing and improving LLMs for TSN applications. First, models should be trained on TSN documents, such as the IEEE 802.1 standards, to strengthen their domain knowledge.

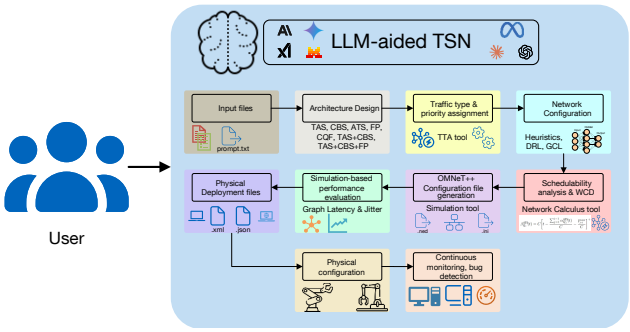


Fig. 4: A vision and a conceptual framework illustrating LLM-assisted TSN management and deployment.

To further facilitate the configuration of TSN hardware and switches, company-specific documentation can be incorporated through retrieval-augmented generation (RAG) or, where appropriate, model fine-tuning, enabling network engineers to interact with LLMs and receive configuration guidance for new TSN hardware deployments. This approach would ultimately reduce manual effort and operational overhead. However, LLM outputs cannot be used as-is without appropriate verification. Therefore, we must develop new tools or leverage existing ones to verify LLM outputs before deploying them in real-world scenarios. Existing mathematical models [16], [17], [26], heuristic and metaheuristic-based algorithms [9], [10], [24], NC tools [18]–[20], [85], and ML-based solutions [25], [29], [30] provide strong foundations for this verification. LLMs can generate code for NC-based evaluation, constraints for scheduling and configuration algorithms, code for generating the final output, and network simulation files. However, providing the necessary guarantees for LLM outputs requires additional tools to verify them. Automated verification is therefore crucial for utilizing LLMs effectively in TSN management and orchestration frameworks (see Finding 14).

Finding 14: LLM-generated configurations cannot be directly deployed for TSN management without external verification from traditional tools.

VII. Research Roadmap Toward Deployable TSN

To make TSN deployable, there are many open research directions. Here, we summarize the identified research directions for the short, medium, and long term.

Short-term: In the short term, research should focus on developing AI-assisted scheduling and configuration algorithms that account for hardware limitations while generating solutions. Lifecycle-aware scheduling and configuration represent the first critical step toward making TSN deployable.

Medium-term: Over the next four to five years, research should focus on developing automated TSN deployment pipeline architectures. AI-assisted verification and configuration tools that take natural language as input and generate configuration and scheduling tables, verified by mathematical models, represent the future of TSN network management. Such tools will assist network engineers with deployment,

reducing the burden on domain experts. Runtime reconfiguration is not a new concept, yet a fully deployable, runtime-reconfigurable solution for all stages of the TSN lifecycle remains out of reach.

Long-term: The long-term goal is to develop an end-to-end AI-assisted TSN deployment pipeline that provides verifiable solutions and configuration artifacts, uses mathematical models to verify generated solutions, automatically monitors the deployed network, and assists with runtime reconfiguration. Developing agentic-TSN, where network devices can self-configure and self-heal to support a plug-and-play architecture, is essential for modern communication infrastructure. Moreover, the standardization community must work toward providing standardized datasets, benchmarks, and, most importantly, guidelines for the unified development of configuration, scheduling, and optimization solutions.

VIII. Conclusion

Deploying TSN is an end-to-end engineering problem that extends well beyond the design of individual mechanisms. This paper examined the TSN deployment lifecycle and identified where current practice falls short. Existing tools ignore shaper interactions and hardware constraints. Industrial profiles select mechanisms without resolving their configuration. Runtime reconfiguration lacks bounded and consistent update paths. AI-assisted management remains trustworthy only when gated by deterministic verification. Closing these gaps requires treating configuration, evaluation, and automation as stages of a single workflow rather than isolated research problems. Realizing such a workflow demands community infrastructure that includes open tools, reference benchmarks, and transparent guidance on which mechanism belongs in which use case. The long-term goal is agentic-TSN networks that self-configure and self-heal while remaining verifiable through mathematical analysis. In future work, we will develop the framework and requirements for agentic-TSN toward adaptive, runtime-reconfigurable TSN deployment.

References

[1] M. Ashjaei, L. L. Bello, M. Daneshtalab, G. Patti, S. Saponara, and S. Mubeen, "Time-Sensitive Networking in automotive embedded systems: State of the art and research opportunities," *Journal of Systems Architecture*, vol. 117, p. 102137, 2021.

[2] T. Zhang, G. Wang, C. Xue, J. Wang, M. Nixon, and S. Han, "Time-Sensitive Networking (TSN) for Industrial Automation: Current Advances and Future Directions," *ACM Comput. Surv.*, vol. 57, no. 2, 2024.

[3] T. Fiori, F. G. Lavacca, F. Valente, and V. Eramo, "Proposal and Investigation of a Lite Time Sensitive Networking Solution for the Support of Real Time Services in Space Launcher Networks," *IEEE Access*, vol. 12, pp. 10 664–10 680, 2024.

[4] Y. Lu, G. Zhao, C. Chakraborty, C. Xu, L. Yang, and K. Yu, "Time-Sensitive Networking-Driven Deterministic Low-Latency Communication for Real-Time Telemedicine and e-Health Services," *IEEE Transactions on Consumer Electronics*, vol. 69, no. 4, 2023.

[5] IEEE, "IEEE Standard for Local and metropolitan area networks – Bridges and Bridged Networks - Amendment 25: Enhancements for Scheduled Traffic," *IEEE Std 802.1Qbv-2015*, pp. 1–57, 2016.

[6] ——, "IEEE Standard for Local and metropolitan area networks–Bridges and Bridged Networks–Amendment 29: Cyclic Queuing and Forwarding," *IEEE 802.1Qch-2017*, pp. 1–30, 2017.

[7] M. Chen, X. Geng, Z. Li, J. Joung, and J. Ryoo, "Segment Routing (SR) Based Bounded Latency," Internet Engineering Task Force, Internet-Draft draft-chen-detnet-sr-based-bounded-latency-03, Jul. 2023, Work in Progress. [Online]. Available: https://datatracker.ietf.org/doc/draft-chen-detnet-sr-based-bounded-latency/03/

[8] N. Finn, "Multiple Cyclic Queuing and Forwarding," https://www.ieee802.org/1/files/public/docs2019/df-finn-multiple-CQF-0919-v02.pdf, 2019, [Accessed 24-February-2024].

[9] K. Alexandris, P. Pop, and T. Wang, "Configuration and Evaluation of Multi-CQF Shapers in IEEE 802.1 Time-Sensitive Networking (TSN)," *IEEE Access*, vol. 10, pp. 109 068–109 081, 2022.

[10] R. Debnath, M. Barzegaran, and S. Steinhorst, "Toward an optimized multi-cyclic queuing and forwarding in time-sensitive networking with time injection," *IEEE Internet of Things Journal*, vol. 12, no. 20, 2025.

[11] IEEE, "IEEE Standard for Local and Metropolitan Area Networks - Virtual Bridged Local Area Networks Amendment 12: Forwarding and Queuing Enhancements for Time-Sensitive Streams," *IEEE Std 802.1Qav-2009*, pp. C1–72, 2010.

[12] ——, "IEEE Standard for Local and Metropolitan Area Networks–Bridges and Bridged Networks - Amendment 34:Asynchronous Traffic Shaping," *IEEE Std 802.1Qcr-2020*, pp. 1–151, 2020.

[13] J. Specht and S. Samii, "Urgency-based scheduler for time-sensitive switched ethernet networks," in *2016 28th Euromicro Conference on Real-Time Systems (ECRTS)*, 2016.

[14] IEEE, "IEEE Standard for Local and metropolitan area networks – Bridges and Bridged Networks – Amendment 26: Frame Preemption," *IEEE Std 802.1Qbu-2016*, pp. 1–52, 2016.

[15] R. Debnath, P. Hortig, L. Zhao, and S. Steinhorst, "Quantifying the Impact of Frame Preemption on Combined TSN Shapers," in *Proc. NOMS*, 2024, pp. 1–9.

[16] S. S. Craciunas, R. Serna Oliver, M. Chmelík, and W. Steiner, "Scheduling Real-Time Communication in IEEE 802.1Qbv Time Sensitive Networks," in *Proc. RTNS*. ACM, 2016.

[17] J. Falk, F. Dürr, and K. Rothermel, "Exploring Practical Limitations of Joint Routing and Scheduling for TSN with ILP," in *Proc. RTCSA*, 2018.

[18] L. Zhao, P. Pop, Z. Zheng, and Q. Li, "Timing Analysis of AVB Traffic in TSN Networks Using Network Calculus," in *Proc. RTAS*, 2018.

[19] L. Zhao, P. Pop, Z. Zheng, H. Daigmorte, and M. Boyer, "Latency Analysis of Multiple Classes of AVB Traffic in TSN With Standard Credit Behavior Using Network Calculus," *IEEE Transactions on Industrial Electronics*, vol. 68, no. 10, 2021.

[20] L. Zhao, P. Pop, and S. Steinhorst, "Quantitative Performance Comparison of Various Traffic Shapers in Time-Sensitive Networking," *IEEE Transactions on Network and Service Management*, vol. 19, no. 3, 2022.

[21] P. Pop, K. Alexandris, and T. Wang, "Configuration of multi-shaper Time-Sensitive Networking for industrial applications," *IET Networks*, vol. 13, no. 5-6, 2024.

[22] R. Debnath, P. Hortig, L. Zhao, and S. Steinhorst, "Advanced Modeling and Analysis of Individual and Combined TSN Shapers in OMNeT++," in *Proc. RTCSA*, 2023.

[23] V. Gavriluţ and P. Pop, "Analysis and Optimisation of Multishaper Multidomain Time-Sensitive Networks: State-of-the-Art and Open Challenges," *IET Cyber-Phys. Syst.: Theory Appl.*, vol. 11, no. 1, 2026.

[24] V. Gavriluţ and P. Pop, "Traffic-Type Assignment for TSN-Based Mixed-Criticality Cyber-Physical Systems," *ACM Trans. Cyber-Phys. Syst.*, vol. 4, no. 2, Jan 2020.

[25] R. Debnath, L. Zhao, and S. Steinhorst, "Learning-Based Traffic Classification for Mixed-Critical Flows in Time-Sensitive Networking," in *Proc. ICC*, 2025.

[26] R. Serna Oliver, S. S. Craciunas, and W. Steiner, "IEEE 802.1Qbv Gate Control List Synthesis Using Array Theory Encoding," in *Proc. RTAS*, 2018.

[27] N. Reusch, L. Zhao, S. S. Craciunas, and P. Pop, "Window-Based Schedule Synthesis for Industrial IEEE 802.1Qbv TSN Networks," in *Proc. WFCS*, 2020, pp. 1–4.

[28] D. Guidolin-Pina, M. Boyer, and J.-Y. Le Boudec, "Work in Progress: Configuration of Cycle Duration in Cyclic Queuing and Forwarding," in *Proc. RTAS*, 2026.

[29] X. He, X. Zhuge, F. Dang, W. Xu, and Z. Yang, "DeepScheduler: Enabling Flow-Aware Scheduling in Time-Sensitive Networking," in *Proc. INFOCOM*, 2023.

[30] Z. Cheng, D. Yang, W. Zhang, J. Ren, H. Wang, and H. Zhang, "DeepCQF: Making CQF Scheduling More Intelligent and Practicable," in *Proc. ICC*, 2022.

[31] J. Yan, W. Quan, X. Jiang, and Z. Sun, "Injection Time Planning: Making CQF Practical in Time-Sensitive Networking," in *INFOCOM*, 2020.

[32] R. Debnath, M. S. Akinci, D. Ajith, and S. Steinhorst, "5GTQ: QoS-Aware 5G-TSN Simulation Framework," in *Proc. VTC2023-Fall*, 2023.

[33] M. Ulbricht *et al.*, "TSN-FlexTest: Flexible TSN Measurement Testbed," *IEEE TNSM*, vol. 21, no. 2, pp. 1387–1402, 2024.

[34] R. Debnath, L. Zhao, M. Barzegaran, and S. Steinhorst, "CyclicSim: Comprehensive Evaluation of Cyclic Shapers in Time-Sensitive Networking," in *Proc. CCNC*, 2025.

[35] S. Bhattacharjee, K. Alexandris, and T. Bauschert, "Multi-Domain TSN Orchestration and Management for Large-Scale Industrial Networks," *IEEE Transactions on Network and Service Management*, vol. 21, 2024.

[36] C. Gärtner, A. Rizk, B. Koldehofe, R. Guillaume, R. Kundel, and R. Steinmetz, "Fast Incremental Reconfiguration of Dynamic Time-Sensitive Networks at Runtime," *Computer Networks*, vol. 224, 2023.

[37] A. Pruski, M. A. Ojewale, V. Gavriluţ, P. M. Yomsi, M. S. Berger, and L. Almeida, "Implementation Cost Comparison of TSN Traffic Control Mechanisms," in *Proc. ETFA*, 2021.

[38] "IEC/IEEE International Standard Time-Sensitive Networking Profile for Industrial Automation," *IEC/IEEE 60802 Edition 1.0 2026-06*, 2026.

[39] "IEEE Standard for Local and metropolitan area networks - Time-Sensitive Networking Profile for Automotive In-Vehicle Ethernet Communications," *IEEE Std 802.1DG-2025*, pp. 1–62, 2025.

[40] A. Ademaj and T. Enzinger, "Traffic Type Characterization for Time-Sensitive Networking," https://www.ieee802.org/1/files/public/docs2018/60802-ademaj-traffic-type-characterization-1118-v01.pdf, 2018, [Online; Accessed 1-May-2023].

[41] I. I. Consortium, "IIC TSN Testbed Traffic Whitepaper," https://www.iiconsortium.org/pdf/IIC_TSN_Testbed_Traffic_Whitepaper_20180418.pdf, 2018, [Online; Accessed 1-May-2023].

[42] ——, "Time Sensitive Networks for Flexible Manufacturing Testbed – Characterization and Mapping of Converged Traffic Types," Version 1.0, https://www.iiconsortium.org/pdf/IIC_TSN_Testbed_Char_Mapping_of_Converged_Traffic_Types_Whitepaper_20180328.pdf, 2019, [Online; Accessed 10-July-2026].

[43] IEEE, "IEEE Standard for Local and metropolitan area networks–Frame Replication and Elimination for Reliability," *IEEE Std 802.1CB-2017*.

[44] M. Turner, S. Lachner, and L. Völker, "What You Need To Know About TSN," TSN Training, 9th Automotive Ethernet Congress, Munich, https://some-ip.com/tsn-training-aec-2023/2023_AEC-TSN-Workshop-final_v1.pdf, 2023, [Online; Accessed 9-7-2026].

[45] M. Ashjaei, G. Patti, M. Behnam, T. Nolte, G. Alderisi, and L. Lo Bello, "Schedulability analysis of ethernet audio video bridging networks with scheduled traffic support," *Real-Time Syst.*, vol. 53, no. 4, Jul. 2017.

[46] G. Janker, "Comparison of Traffic Delay Measurements for a Simple Automotive Topology," https://www.ieee802.org/1/files/public/docs2021/dg-janker-delay-measuement-results-automotive_0621-v01-1.pdf, 2021, [Online; Accessed 10-July-2026].

[47] M. Jochim, "What are the Goals of 802.1 TSN? How can we achieve them?" IEEE 802.1 TSN contribution, https://www.ieee802.org/1/files/public/docs2013/new-tsn-jochim-goals-of-802-1tsn-0713-v01.pdf, 2013, [Online; Accessed 10-July-2026].

[48] D. Pannell, "Choosing the Right TSN Tools to Meet a Bounded Latency," IEEE P802.1DG contribution, https://www.ieee802.org/1/files/public/docs2021/dg-pannell-ChoosingTheRightTSNToolsToMeetABoundedLatency-whitePaper-0821-v01.pdf, 2021, [Online; Accessed 10-July-2026].

[49] K. Budweiser, "An Overview on a Risk-Averse Introduction of TSN Features in Automotive Networks," IEEE P802.1DG contribution, https://www.ieee802.org/1/files/public/docs2021/dg-budweiserBMW-RiskaverseChoiceForTSN-1021-v02.pdf, 2021.

[50] K. Budweiser and M. Heinzinger, "Proposal for an "AVB Profile for Bridged Automotive Ethernet Networks" as used by BMW," IEEE P802.1DG contribution, https://www.ieee802.org/1/files/public/docs2022/dg-budweiser-AVBProfile-0322-v02.pdf, 2022, [Online; Accessed 10-July-2026].

[51] S. S. Craciunas and R. Oliver Serna, "Combined task- and network-level scheduling for distributed time-triggered systems," *Real-Time Syst.*, vol. 52, no. 2, pp. 161–200, Mar. 2016.

[52] L. Chen and Y. Zhang, "Automotive In-Vehicle Traffic Type," IEEE P802.1DG contribution, https://www.ieee802.org/1/files/public/docs2019/dg-chen-automotive-traffic-type-0719-v01.pdf, 2019, [Online; Accessed 10-July-2026].

[53] M. Turner, "IEEE 802.1DG – Traffic Categories," https://www.ieee802.org/1/files/public/docs2020/dg-turner-traffic-classification-0820-v01.pdf, 2020, [Online; Accessed 12-July-2026].

[54] A. Gopal, "Traffic Categories & Overall Performance Goals," IEEE P802.1DG contribution, https://www.ieee802.org/1/files/public/docs2022/dg-gopal-TrafficClassification-0222-v01.pdf, 2022, [Online; Accessed 10-July-2026].

[55] J. Specht and S. Samii, "Urgency Based Scheduler – Automotive Use," IEEE 802.1 TSN contribution, https://www.ieee802.org/1/files/public/docs2014/new-tsn-specht-ubs-automotive-1114-v01.pdf, 2014, [Online; Accessed 12-July-2026].

[56] L. Lo Bello and W. Steiner, "A Perspective on IEEE Time-Sensitive Networking for Industrial Communication and Automation Systems," *Proceedings of the IEEE*, vol. 107, no. 6, pp. 1094–1120, 2019.

[57] M. Becker, D. Dasari, S. Mubeen, M. Behnam, and T. Nolte, "End-to-end timing analysis of cause-effect chains in automotive embedded systems," *J. Syst. Archit.*, vol. 80, no. C, 2017.

[58] M. Dürr, G. V. D. Brüggen, K.-H. Chen, and J.-J. Chen, "End-to-end timing analysis of sporadic cause-effect chains in distributed systems," *ACM Trans. Embed. Comput. Syst.*, vol. 18, no. 5s, Oct. 2019.

[59] S. S. Craciunas, M. Becker, and P. Pop, "Automotive cause-effect chains in the wild: A real-world industrial benchmark and generator," 2026, presented at RTSOPS. [Online]. Available: https://doi.org/10.5281/zenodo.20637979

[60] M. Vlk, K. Brejchová, Z. Hanzálek, and S. Tang, "Large-scale periodic scheduling in time-sensitive networks," *Computers & Operations Research*, vol. 137, p. 105512, 2022.

[61] C. Xue, T. Zhang, Y. Zhou, M. Nixon, A. Loveless, and S. Han, "Real-Time Scheduling for 802.1Qbv Time-Sensitive Networking (TSN): A Systematic Review and Experimental Study," in *Proc. RTAS*, 2024.

[62] A. Bouillard, L. Jouhet, and E. Thierry, "Tight performance bounds in the worst-case analysis of feed-forward networks," in *INFOCOM*, 2010.

[63] P. Pop, M. L. Raagaard, S. S. Craciunas, and W. Steiner, "Design optimisation of cyber-physical distributed systems using IEEE Time-sensitive Networks," *IET Cyber-Phys. Syst.: Theory Appl.*, vol. 1, 2016.

[64] "IEEE Standard for Local and Metropolitan Area Networks–Bridges and Bridged Networks – Amendment 31: Stream Reservation Protocol (SRP) Enhancements and Performance Improvements," *IEEE Std 802.1Qcc-2018*, 2018.

[65] D. Cavalcanti *et al.*, "Common Test Specification for IEC/IEEE 60802: Status Update," Liaison to IEEE 802.1, https://www.ieee802.org/1/files/public/docs2025/liaison-TIACC-CommonTestSpecUpdate-0325.pdf, 2025, [Online; Accessed 11-July-2026].

[66] IEEE, "IEEE Standard for Local and metropolitan area networks–Audio Video Bridging (AVB) Systems," *IEEE Std 802.1BA-2011*, 2011.

[67] ——, "IEEE Standard for Local and Metropolitan Area Networks–Bridges and Bridged Networks," *IEEE Std 802.1Q-2022 (Revision of IEEE Std 802.1Q-2018)*, 2022.

[68] H. Chahed and A. Kassler, "TSN Network Scheduling—Challenges and Approaches," *Network*, vol. 3, no. 4, pp. 585–624, 2023.

[69] IEEE, "P802.1Qdq – Standard for Local and Metropolitan Area Networks–Bridges and Bridged Networks: Amendment: Shaper Parameter Settings for Bursty Traffic Requiring Bounded Latency," Ongoing IEEE 802.1 TSN project, https://1.ieee802.org/tsn/802-1qdq/, 2026, [Online; Accessed 11-July-2026].

[70] M. Boyer and R. Henia, "Embedded Reconfiguration of TSN (Industrial Challenge Description)," in *Proc. ECRTS*, ser. LIPIcs, vol. 335, 2025.

[71] Z. Pang *et al.*, "Flow Scheduling for Conflict-Free Network Updates in Time-Sensitive Software-Defined Networks," *IEEE Trans. Ind. Inform.*, vol. 17, 2021.

[72] W. Kong, M. Nabi, and K. G. W. Goossens, "Run-Time Recovery and Failure Analysis of Time-Triggered Traffic in Time Sensitive Networks," *IEEE Access*, vol. 9, pp. 91 710–91 722, 2021.

[73] M. Boyer, R. Henia, and C. Pralet, "Updating TAS schedule without reconfiguring gates in TSN," in *Proc. RTNS*, 2025.

[74] P.-J. Chaine and M. Boyer, "Shortening gate closing time to limit bandwidth waste when implementing Time-Triggered scheduling in TAS/TSN," in *Proc. JRWRTC*, Paris, France, 2022.

[75] C. Gärtner, A. Rizk, B. Koldehofe, R. Hark, R. Guillaume, and R. Steinmetz, "Leveraging flexibility of time-sensitive networks for dynamic reconfigurability," in *Proc. IFIP Networking*, 2021.

[76] N. Reusch, M. Barzegaran, L. Zhao, S. S. Craciunas, and P. Pop, "Configuration optimization for heterogeneous time-sensitive networks," *Real-Time Syst.*, vol. 59, no. 4, Nov. 2023.

[77] W. Sun, Y. Zou, N. Guan, X. Zhang, G. Du, and Y. Wen, "Graph Attention Network–Based Deep Reinforcement Learning Scheduling Framework for in-Vehicle Time-Sensitive Networking," *IEEE Transactions on Industrial Informatics*, vol. 20, no. 7, pp. 9825–9836, 2024.

[78] H. Yu, T. Taleb, and J. Zhang, "Deep Reinforcement Learning based Deterministic Routing and Scheduling for Mixed-Criticality Flows," *IEEE Transactions on Industrial Informatics*, pp. 1–11, 2022.

[79] S. Long *et al.*, "A Survey on Intelligent Network Operations and Performance Optimization Based on Large Language Models," *IEEE Communications Surveys & Tutorials*, vol. 27, no. 6, 2025.

[80] P. Sharma and V. Yegneswaran, "PROSPER: Extracting Protocol Specifications Using Large Language Models," in *Proc. HotNets*, 2023.

[81] R. Mondal, A. Tang, R. Beckett, T. Millstein, and G. Varghese, "What do LLMs need to Synthesize Correct Router Configurations?" in *Proc. HotNets*. New York, NY, USA: ACM, 2023.

[82] C. Wang, M. Scazzariello, A. Farshin, S. Ferlin, D. Kostić, and M. Chiesa, "NetConfEval: Can LLMs Facilitate Network Configuration?" *Proc. ACM Netw.*, vol. 2, no. CoNEXT2, Jun. 2024.

[83] R. Debnath, L. Zhao, M. Barzegaran, P. Pop, and S. Steinhorst, "Introducing Large Language Models into the Design Flow of Time Sensitive Networking," in *Proc. VTC2026-Fall*, 2026.

[84] R. Debnath, D. B. Mateu, L. Zhao, S. S. Craciunas, P. Pop, and S. Steinhorst, "TSNBench: Benchmarking LLM Proficiency in Time-Sensitive Networking," 2026. [Online]. Available: https://arxiv.org/abs/2605.09481

[85] L. Zhao, L. Rao, Q. Li, and R. Debnath, "Formal Timing Analysis of CQF Interference in TSN: A Network Calculus-Based Approach," in *Proc. RTSS*, 2025.